

Universidad de la Fuerzas Armadas - ESPE

Departamento de Ciencias de la Computación

Carrera de Ingeniería de Software



Análisis y Diseño de Software - NRC30723

Proyecto: Sistema Bocatto

Título: GUÍA PARA DESARROLLAR EN PLANTUML
DIAGRAMAS DE CASOS DE USO DE ANÁLISIS.

Integrantes: Carlos Kaiza, Kerlly Chiriboga, Alexander Villacres

Tutor: Mgtr. Jenny Ruiz

Fecha de entrega: 4 de Mayo, 2026

GUÍA PARA DESARROLLAR EN PLANTUML DIAGRAMAS DE CASOS DE USO DE ANÁLISIS

Asignatura: Análisis y Diseño de Software

Tema: Modelado de casos de uso de nivel 0 y apoyo al análisis orientado a objetos con PlantUML

Caso base: Gestión de requisitos funcionales según el ERS de Sistema BOCATTO.

CASO DE ESTUDIO: SISTEMA BOCATTO

1. Propósito de la Guía

Esta guía orienta el desarrollo de diagramas de casos de uso de análisis en PlantUML, tomando como referencia el ERS del proyecto Bocatto para transformar los requisitos funcionales de nivel 0 en casos de uso claros, trazables y verificables antes de modelar su estructura interna..

2. Requisitos funcionales de nivel 0

Código	Requisito Funcional	Actor Principal	Caso de Uso
RF-00	Acceso seguro mediante usuario y clave.	Usuario (Personal)	CU0-0 Autenticar en el sistema
RF-01	Registro de pedidos directamente en la mesa.	Mesero	CU0-1 Tomar pedido en mesa
RF-02	Toma de pedidos vía telefónica para delivery.	Operador / Recepcionista	CU0-2 Tomar pedido telefónico
RF-03	Autogestión de pedido mediante código QR.	Cliente	CU0-3 Realizar pedido por QR

3. Diagrama de Casos de Uso (PlantUML)

Representación del límite del sistema y los actores (roles externos) y los casos de uso involucrados en los procesos de pedido.

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false

actor "Mesero" as M
actor "Operador" as O
actor "Cliente" as C
actor "Cocinero" as CO

rectangle "Sistema Bocatto - Gestión de Pedidos" {
usecase "CU0-0: Autenticar en el sistema" as UC0
usecase "CU0-1: Tomar pedido en mesa" as UC1
```

```

usecase "CU0-2: Tomar pedido telefónico" as UC2
usecase "CU0-3: Realizar pedido por QR" as UC3
usecase "Validar datos del pedido" as UC_Val
usecase "CU0-4: Gestionar preparación del pedido" as UC4

usecase "Revisar lista de pedidos pendientes" as UC4_Review
usecase "Seleccionar y actualizar estado del pedido" as UC4_Update
}

M --> UC1
O --> UC2
C --> UC3
CO -right-> UC4

UC1 ..> UC_Val : <<include>>
UC2 ..> UC_Val : <<include>>
UC3 ..> UC_Val : <<include>>

UC1 ..> UC0 : <<include>>
UC2 ..> UC0 : <<include>>
UC4 ..> UC0 : <<include>>

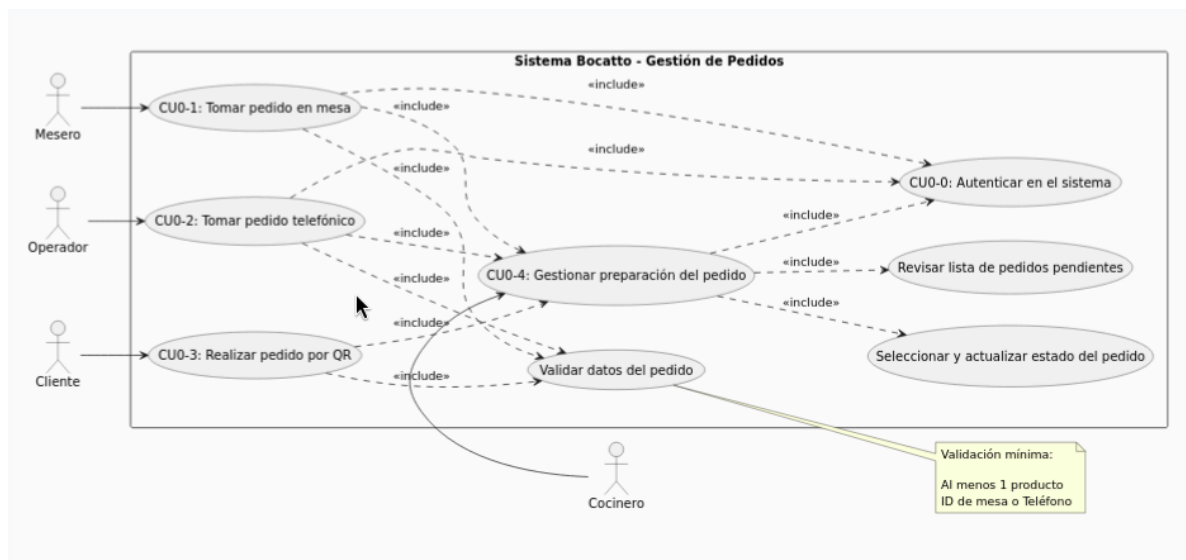
' CU0-1..CU0-3 include CU0-4 (preparation management is part of taking orders)
UC1 ..> UC4 : <<include>>
UC2 ..> UC4 : <<include>>
UC3 ..> UC4 : <<include>>

' CU0-4 composition
UC4 ..> UC4_Review : <<include>>
UC4 ..> UC4_Update : <<include>>

note right of UC_Val
Validación mínima:
Al menos 1 producto
ID de mesa o Teléfono
end note

@enduml

```



4. Especificación breve de casos de uso

Caso de uso	Actor	Precondición	Flujo principal resumido	Resultado esperado
CU0-0: Autenticar en el sistema	Usuario (Personal autorizado)	El usuario debe estar registrado previamente con un rol asignado.	Ingresar credenciales -> validar acceso -> identificar rol -> habilitar funcionalidades.	Acceso concedido y menú mostrado según el rol del usuario.
CU0-1: Tomar pedido en mesa	Mesero	El mesero debe haber iniciado sesión correctamente.	Seleccionar mesa y platos -> validar datos del pedido -> registrar orden -> generar identificador único.	Pedido registrado con estado "pendiente" y enviado a cocina.
CU0-2: Tomar pedido telefónico	Operador	El operador debe estar autenticado en su estación de trabajo.	Ingresar teléfono del cliente -> validar cliente -> seleccionar productos -> confirmar registro.	Pedido para delivery registrado con estado inicial "pendiente".
CU0-3: Realizar pedido por QR	Cliente	El cliente debe contar con un dispositivo móvil con cámara y conexión a internet.	Escanear código QR -> visualizar menú digital -> seleccionar productos -> confirmar carrito -> enviar orden.	Pedido autogestionado correctamente y enviado a cocina para preparación.

5. Puente hacia clases de análisis: frontera, control y entidad

Tipo de clase	Propósito en análisis	Ejemplo para Sistema Bocatto	Sintaxis PlantUML sugerida
Frontera / Boundary	Representa la interacción con el actor o una pantalla del sistema.	FormularioLogin, interfazPedidoMesa, AppMovilQR.	boundary "FormularioLogin", "Interfaz Pedido Mesa", "InterfazPedidoTel", "AppMovilQR" as UI.
Control / Control	Coordina la lógica del caso de uso y la comunicación entre frontera y entidad.	ControlRegistroPedido, GestorAutenticacion.	control "Control Registro Pedido", "GestorAutenticacion" as Ctrl.
Entidad / Entity	Representa información	Pedido, Plato, Usuario, Cliente.	entity "Pedido", "Plato",

	del dominio que el sistema conserva.		“Usuario”, “Cliente” as Ent.
Repositorio / Repository	Conserva y recupera registros o colecciones de entidades.	RepositorioUsuarios.	database "Repositorio Usuarios", “RepositorioPedidos” as Repo.

1. Puente para RF-00: Autenticar en el sistema

```

@startuml
left to right direction
actor "Usuario (Personal)" as Actor
boundary "FormularioLogin" as UI
control "GestorAutenticacion" as Ctrl
entity "Usuario" as Ent

Actor --> UI : ingresa credenciales (user/pass)
UI --> Ctrl : validarAcceso(user, pass)
Ctrl --> Ent : verifica existencia y rol
Ctrl --> UI : otorga acceso y muestra menú
UI --> Actor : permite navegación según rol
@enduml

```



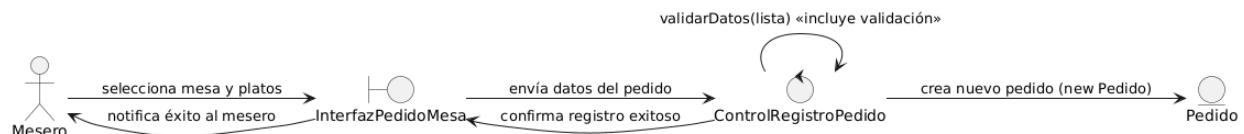
2. Puente para RF-01: Tomar pedido en mesa (Incluye Validación)

```

@startuml
left to right direction
actor "Mesero" as Actor
boundary "InterfazPedidoMesa" as UI
control "ControlRegistroPedido" as Ctrl
entity "Pedido" as Ent

Actor --> UI : selecciona mesa y platos
UI --> Ctrl : envía datos del pedido
Ctrl --> Ctrl : validarDatos(lista) <<incluye validación>>
Ctrl --> Ent : crea nuevo pedido (new Pedido)
Ctrl --> UI : confirma registro exitoso
UI --> Actor : notifica éxito al mesero
@enduml

```



3. Puente para RF-02: Tomar pedido telefónico

```

@startuml

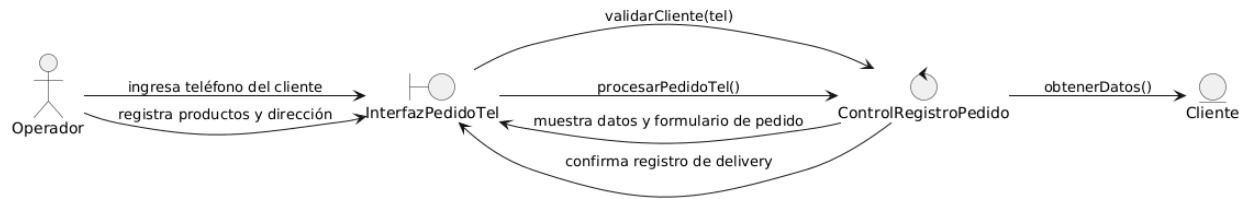
```

```

left to right direction
actor "Operador" as Actor
boundary "InterfazPedidoTel" as UI
control "ControlRegistroPedido" as Ctrl
entity "Cliente" as Ent

Actor --> UI : ingresa teléfono del cliente
UI --> Ctrl : validarCliente(tel)
Ctrl --> Ent : obtenerDatos()
Ctrl --> UI : muestra datos y formulario de pedido
Actor --> UI : registra productos y dirección
UI --> Ctrl : procesarPedidoTel()
Ctrl --> UI : confirma registro de delivery
@enduml

```



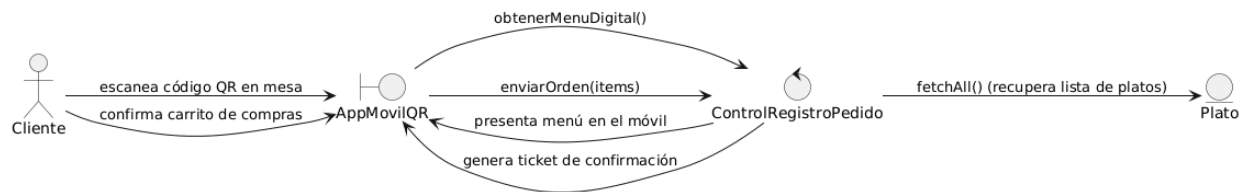
4. Puente para RF-03: Realizar pedido por QR

```

@startuml
left to right direction
actor "Cliente" as Actor
boundary "AppMovilQR" as UI
control "ControlRegistroPedido" as Ctrl
entity "Plato" as Ent

Actor --> UI : escanea código QR en mesa
UI --> Ctrl : obtenerMenuDigital()
Ctrl --> Ent : fetchAll() (recupera lista de platos)
Ctrl --> UI : presenta menú en el móvil
Actor --> UI : confirma carrito de compras
UI --> Ctrl : enviarOrden(items)
Ctrl --> UI : genera ticket de confirmación
@enduml

```



6. Matriz de Trazabilidad RF-CU

RF	Caso de uso	Actor	Prioridad	Criterio de aceptación	Evidencia PlantUML
RF-00	CU0-0	Usuario	Alta	El acceso se otorga solo	UC0

RF	Caso de uso	Actor	Prioridad	Criterio de aceptación	Evidencia PlantUML
				con redenciales válidas registradas.	
RF-01	CU0-1	Mesero	Alta	El sistema vincula el pedido a una mesa física específica.	UC1
RF-02	CU0-2	Operador	Media	Se capturan los datos de contacto y entrega del cliente.	UC2
RF-03	CU0-3	Cliente	Alta	El pedido se genera mediante la lectura del identificador de mesa QR.	UC3

7. Conclusión técnica

El desarrollo del modelo funcional inicial permite establecer una trazabilidad vertical sólida, asegurando que cada Requisito Funcional (RF) de nivel 0 está directamente vinculado a un Caso de Uso (CU) y un criterio de aceptación claro; esto garantiza que el sistema no incluya funcionalidades innecesarias ni omita necesidades del negocio. La consistencia del modelo se alcanza mediante el uso de una nomenclatura estandarizada (verbo + objeto) y la delimitación precisa de las fronteras del sistema, lo que asegura que la interacción entre los actores externos y la solución de software sea lógica y coherente.

Finalmente, la utilidad de este análisis radica en su función como "puente" hacia el diseño estructural, proporcionando la base necesaria para identificar las clases, atributos y responsabilidades que darán vida al sistema en las etapas posteriores del desarrollo.

8. Fuentes internas consultadas

Documentos revisados desde la carpeta de Google Drive compartida por la docente:

- Modelado de Clases de Análisis en PlantUML.docx
- Tareas_U1_Autonomo_Analisis_Diseño_OO_PlantUML.docx